DEEP VISION CONSULTING

UNIMORE

# Image Processing

## Intro to OpenCV:

### basic camera, image and video controls

Francesco Solera

October 15th, 2026

# A camera for each application

# A camera for each application





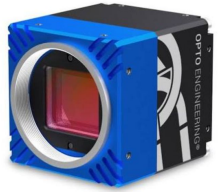





Different applications require different specific camera setups:

# A camera for each application







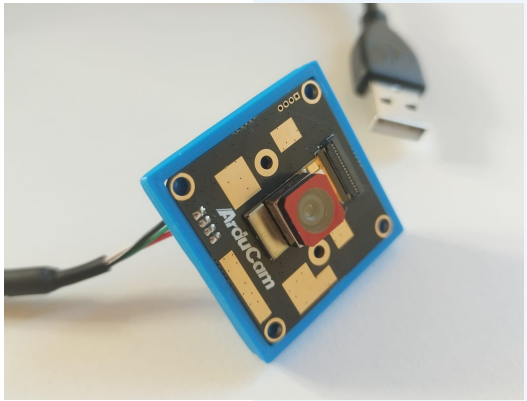
Different applications require different specific camera setups:

- cost
- ruggedness
- image quality
- zoom & focus
- settable parameters
- shape and size

But they share almost the same principles when it comes to *image formation* and *image processing*.

# Lab kit!

EBV Elektronik | An Avnet Company | ArduCam®




Arducam "ezBox" family, but none of the public ones.

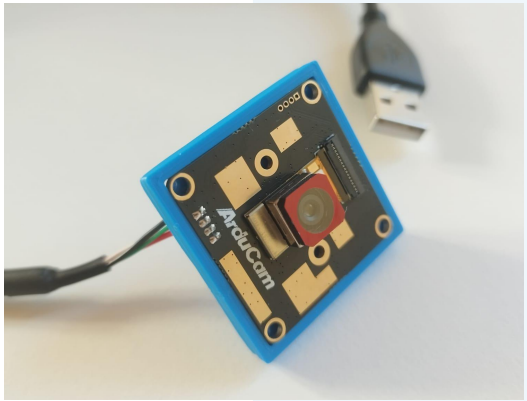
We gratefully acknowledge EBV's and Arducam's support of this course through the donation of a webcam to every student for the laboratory sessions.

EBV, Arducam and DVC sponsored a lab kit for every student of the course.

- Your kit should contain:
  - UVC camera + 3D printed case + sticky pads
  - USB cable
  - cardboards + measuring tape for exercises

# Lab kit!

EBV Elektronik | An Avnet Company | ArduCam®




Arducam "ezBox" family, but none of the public ones.

We gratefully acknowledge EBV's and Arducam's support of this course through the donation of a webcam to every student for the laboratory sessions.

EBV, Arducam and DVC sponsored a lab kit for every student of the course.

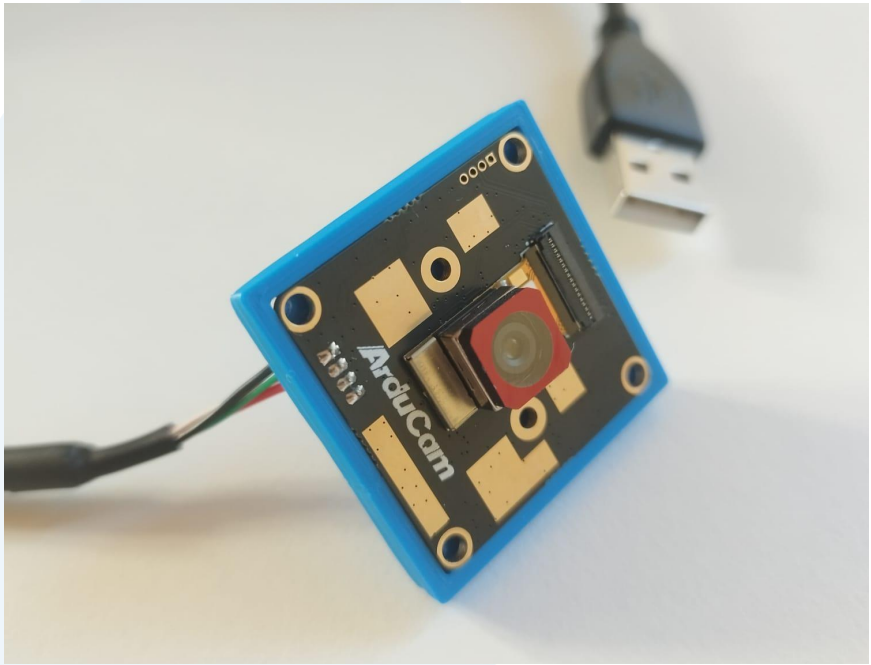
- Your kit should contain:
  - UVC camera + 3D printed case + sticky pads
  - USB cable
  - cardboards + measuring tape for exercises

- By the end of the lecture, you will know how to:
  - set and verify camera configuration;
  - grab, save and show a still image;
  - grab, save and show a short video;
  - estimate camera properties (e.g. focal and FOV)

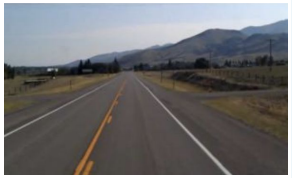
EBV Elektronik
I An Avnet Company I

ArduCam®

DEEP VISION CONSULTING



# OpenCV: image data <-> vision algorithms



camera

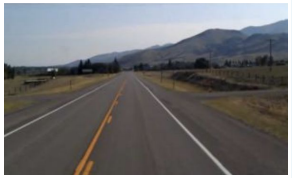
frame / img

???

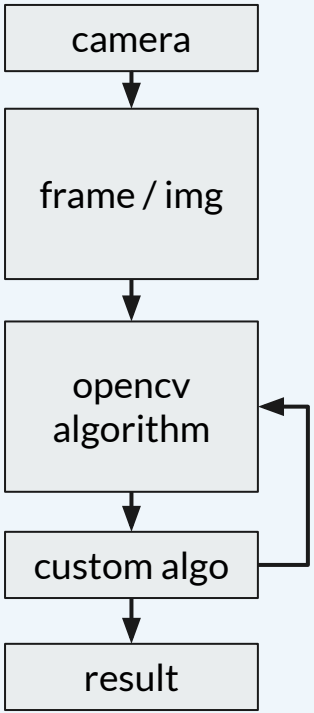
*“go straight”*

result

# OpenCV: image data <-> vision algorithms








*"go straight"*

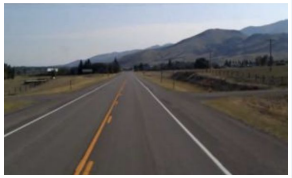
OpenCV is a computer-vision library with:

- open-source C++ code and
- permissive licence.

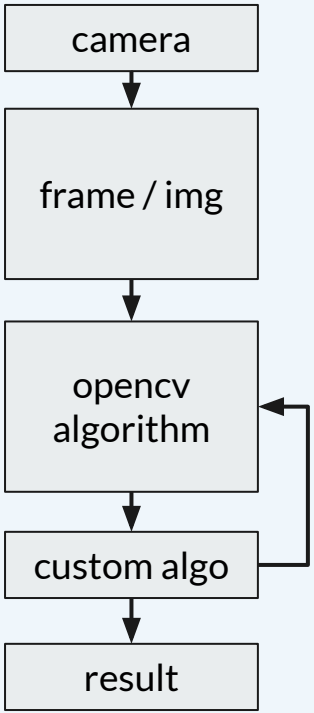
It basically connects 3 important "modules":

- **Images** and frames are arrays.
- **Algorithms** transform these arrays.
- Video **I/O** connects files and cameras.

# OpenCV: image data <-> vision algorithms








*"go straight"*

OpenCV is a computer-vision library with:

- open-source C++ code and
- permissive licence.

It basically connects 3 important "modules":

- **Images** and frames are arrays.
- **Algorithms** transform these arrays.
- Video **I/O** connects files and cameras.

As a plus, it has a nice Python API so we can conveniently use it in both Python and C++: we typically prototype in python and deploy in C++ without having to change the algorithm!

# A frame is a typed numpy array

```
ok, frame = cap.read()

print(frame.shape)   # (H, W, C)
print(frame.dtype)   # usually uint8

blue, green, red = frame[r, c]
```

Very important things to know from day 1:

- **Origin:** top left (pointing right and down)
- **Indexing:** frame[r, c] 0-based
- **Channel order:** BGR (not RGB)
- **Typical range:** 0-255

# A frame is a typed numpy array

```
ok, frame = cap.read()

print(frame.shape)   # (H, W, C)
print(frame.dtype)   # usually uint8

blue, green, red = frame[r, c]
```

Very important things to know from day 1:

- **Origin:** top left (pointing right and down)
- **Indexing:** frame[r, c] 0-based
- **Channel order:** BGR (not RGB)
- **Typical range:** 0-255

To work in float for custom operations:

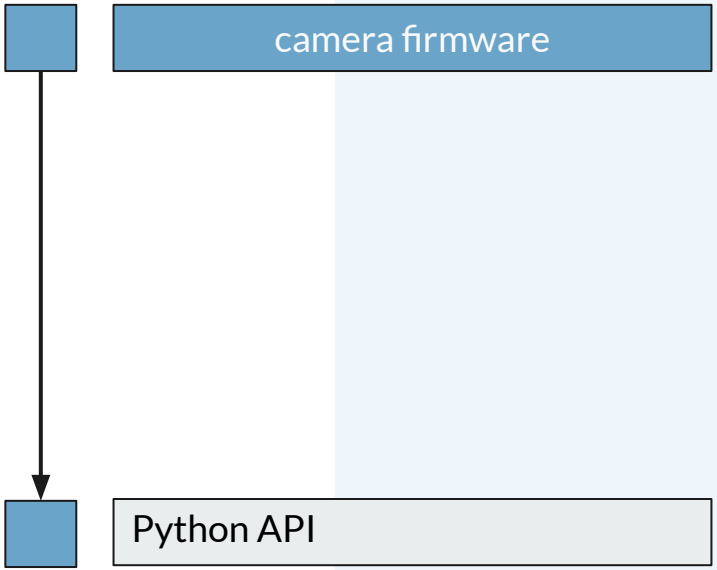
```
frame_float = frame.astype(np.float32) / 255
```

To go back to uint8 for opencv methods:

```
frame = np.clip(frame_float * 255, 0, 255).astype(np.uint8)
```

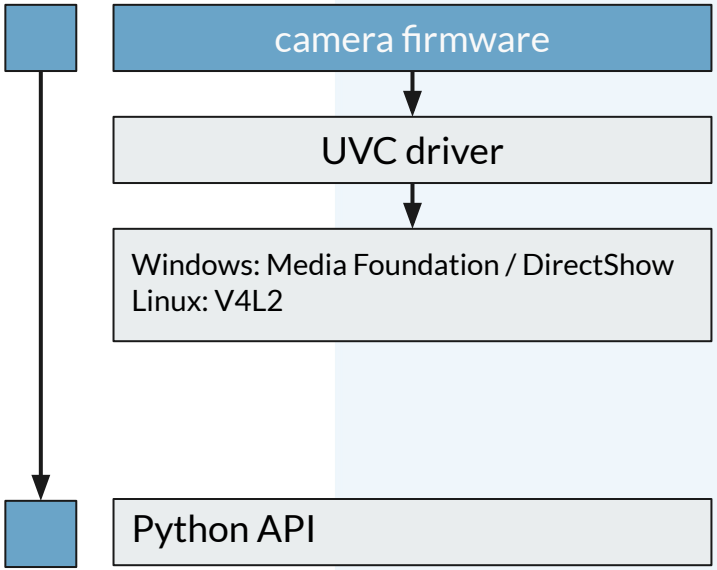
... at least 50% of your bugs will come from this slide!

# UVC as a standard




**Problem:** *A custom camera has its own SDK / python API to interact with the firmware. You change camera -> you must change the code.*

# UVC as a standard




**Problem:** *A custom camera has its own SDK / python API to interact with the firmware. You change camera -> you must change the code.*

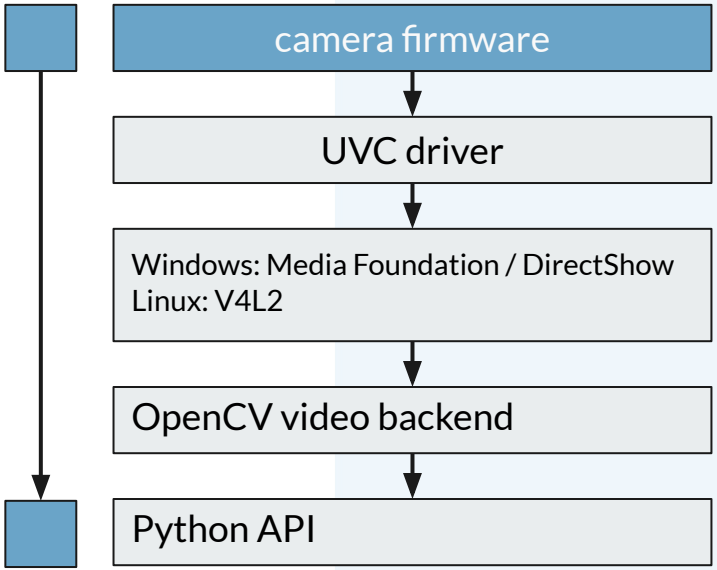
**UVC = USB Video Class**

A UVC camera can describe:

- available video streams;
- resolutions and frame rates;
- pixel or compressed formats;
- supported camera controls and how to set them.

Every capability remains optional and depends on the camera manufacturer.

# UVC as a standard




**Problem:** *A custom camera has its own SDK / python API to interact with the firmware. You change camera -> you must change the code.*

**UVC = USB Video Class**

A UVC camera can describe:

- available video streams;
- resolutions and frame rates;
- pixel or compressed formats;
- supported camera controls and how to set them.

Every capability remains optional and depends on the camera manufacturer.

# What can our UVC camera do?

```
ioctl: VIDIOC_ENUM_FMT
    Type: Video Capture

    [0]: 'YUYV' (YUYV 4:2:2)
        Size: Discrete 640x480
            Interval: Discrete 0.033s (30.000 fps)
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 1920x1080
            Interval: Discrete 0.200s (5.000 fps)
        Size: Discrete 1280x960
            Interval: Discrete 0.067s (15.000 fps)
        Size: Discrete 1280x720
            Interval: Discrete 0.067s (15.000 fps)
        Size: Discrete 800x600
            Interval: Discrete 0.067s (15.000 fps)
        Size: Discrete 320x240
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 640x480
            Interval: Discrete 0.033s (30.000 fps)
            Interval: Discrete 0.033s (30.000 fps)
    [1]: 'MJPG' (Motion-JPEG, compressed)
        Size: Discrete 1920x1080
            Interval: Discrete 0.033s (30.000 fps)
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 1280x960
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 1280x720
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 800x600
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 640x480
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 320x240
            Interval: Discrete 0.033s (30.000 fps)
        Size: Discrete 1920x1080
            Interval: Discrete 0.033s (30.000 fps)
            Interval: Discrete 0.033s (30.000 fps)
```

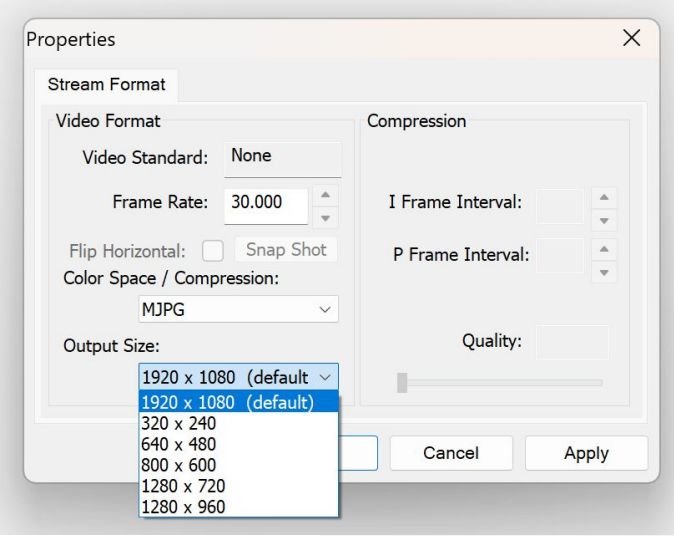
If you're on linux, from terminal:
`v4l2-ctl --device=/dev/video0 --list-formats-ext`

- format
- resolution
- fps

Or for a GUI on Linux, install GUVCView

If you're on windows, download VirtualDub2

# What can our UVC camera do?




If you're on windows:

1. download VirtualDub2
2. navigate the menus:
   A. Capture > Capture AVI...
   B. Device > Arducam-1080P-HDR (DirectShow)
   C. Video > **Capture pin...**

# What can our UVC camera do?

```
User Controls

                     brightness 0x00980900 (int)    : min=-64 max=64 step=1 default=0 value=0
                       contrast 0x00980901 (int)    : min=0 max=64 step=1 default=32 value=32
                     saturation 0x00980902 (int)    : min=0 max=128 step=1 default=64 value=64
                            hue 0x00980903 (int)    : min=-40 max=40 step=1 default=0 value=0
        white_balance_automatic 0x0098090c (bool)   : default=1 value=1
                          gamma 0x00980910 (int)    : min=72 max=500 step=1 default=100 value=100
                           gain 0x00980913 (int)    : min=0 max=100 step=1 default=0 value=0
           power_line_frequency 0x00980918 (menu)   : min=0 max=2 default=2 value=2 (60 Hz)
                0: Disabled
                1: 50 Hz
                2: 60 Hz
white_balance_temperature 0x0098091a (int)    : min=2800 max=6500 step=1 default=4600 value=4600 flags=inactive
                     sharpness 0x0098091b (int)    : min=0 max=6 step=1 default=3 value=3
        backlight_compensation 0x0098091c (int)    : min=0 max=2 step=1 default=1 value=1


Camera Controls

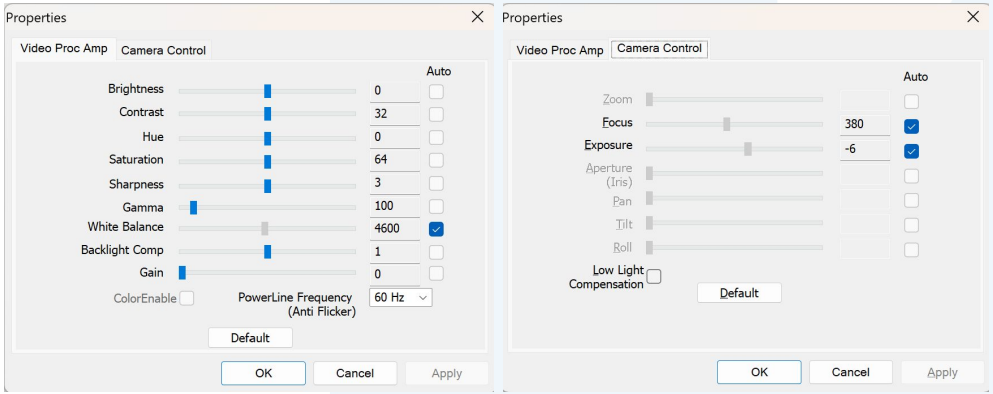
                  auto_exposure 0x009a0901 (menu)   : min=0 max=3 default=3 value=3 (Aperture Priority Mode)
                1: Manual Mode
                3: Aperture Priority Mode
         exposure_time_absolute 0x009a0902 (int)    : min=1 max=5000 step=1 default=157 value=157 flags=inactive
exposure_dynamic_framerate 0x009a0903 (bool)   : default=0 value=0
                 focus_absolute 0x009a090a (int)    : min=1 max=831 step=1 default=1 value=1 flags=inactive
focus_automatic_continuous 0x009a090c (bool)   : default=1 value=1
```

If you're on linux, from terminal:

```
v4l2-ctl --device=/dev/video0
--list-ctrls-menus
```

- focus or autofocus state;
- exposure and gain;
- white balance (more on this later);

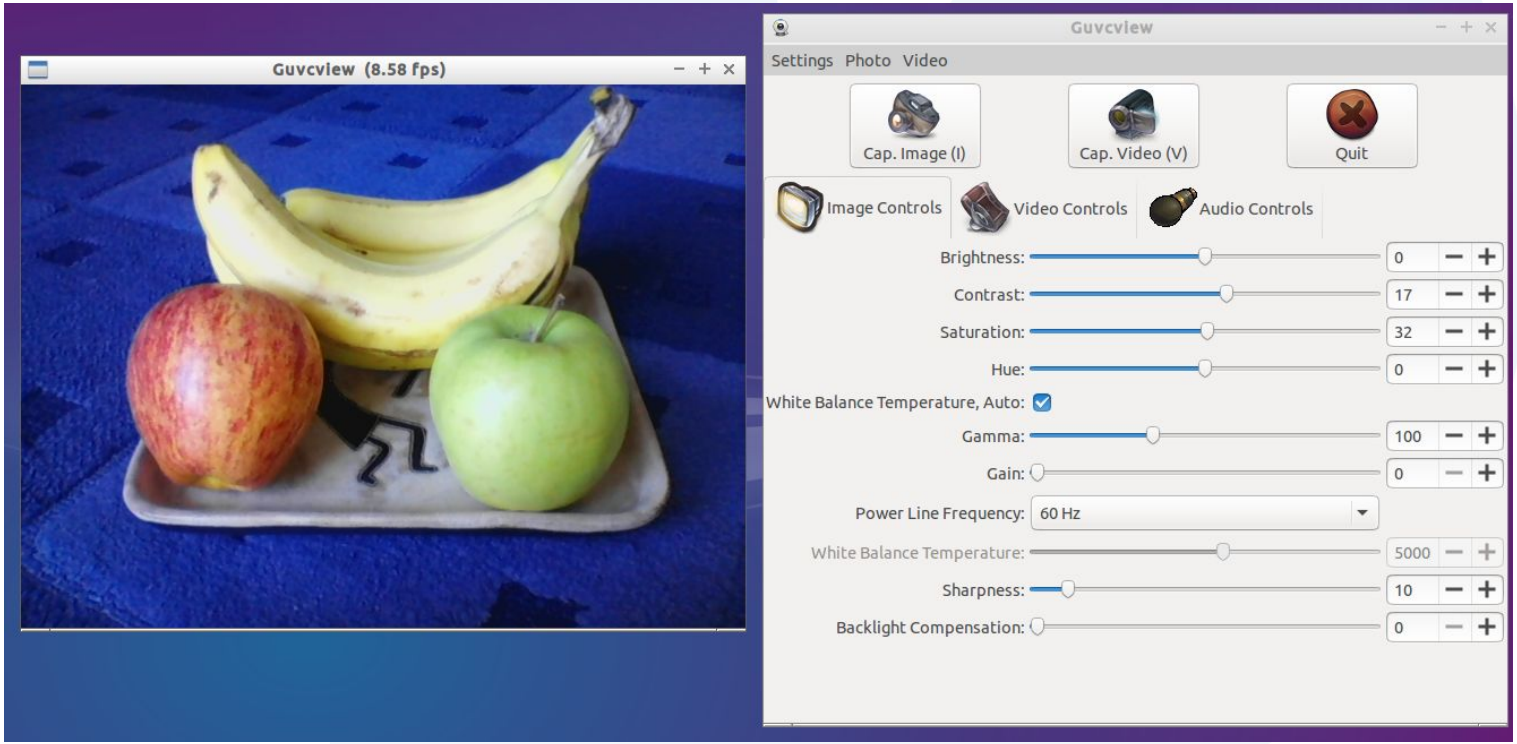
# What can our UVC camera do?




If you're on windows:

1. navigate the menus:
   A. Capture > Capture AVI...
   B. Device > Arducam-1080P-HDR (DirectShow)
   C. Video > **Capture filter...**

# Let’s play a bit with our camera controls



# Lab materials and Python environment

```
image-processing-labs/
├── README.md
├── pyproject.toml
├── uv.lock
├── .python-version
├── .vscode/
│   ├── extensions.json
│   └── launch.json
└── labs/
    ├── __init__.py
    ├── common/
    └── lab03/
        ├── __init__.py
        ├── README.md
        ├── exercises/    # Supplied ex01-03
        └── solutions/    # Complete ex01-05
```

Download the lab ZIP from moodle and extract it.

For subsequent labs:

- use each ZIP independently, or
- copy its `labNN/` folder into your existing labs/ folder to reuse the environment (suggested).

# Lab materials and Python environment

```
image-processing-labs/
├── README.md
├── pyproject.toml
├── uv.lock
├── .python-version
├── .vscode/
│   ├── extensions.json
│   └── launch.json
└── labs/
    ├── __init__.py
    ├── common/
    └── lab03/
        ├── __init__.py
        ├── README.md
        ├── exercises/    # Supplied ex01-03
        └── solutions/    # Complete ex01-05
```

Download the lab ZIP from moodle and extract it.

For subsequent labs:

- use each ZIP independently, or
- copy its `labNN/` folder into your existing labs/ folder to reuse the environment (suggested).

We will use **uv** in this course:

- *Python package and project manager*
- *extremely fast, written in Rust*
- *enables complete reproducibility across systems.*

# Lab materials and Python environment

```
image-processing-labs/
├── README.md
├── pyproject.toml
├── uv.lock
├── .python-version
├── .vscode/
│   ├── extensions.json
│   └── launch.json
└── labs/
    ├── __init__.py
    ├── common/
    └── lab03/
        ├── __init__.py
        ├── README.md
        ├── exercises/    # Supplied ex01-03
        └── solutions/    # Complete ex01-05
```

Download the lab ZIP from moodle and extract it.

For subsequent labs:

- use each ZIP independently, or
- copy its `labNN/` folder into your existing labs/ folder to reuse the environment (suggested).

We will use **uv** in this course:

- *Python package and project manager*
- *extremely fast, written in Rust*
- *enables complete reproducibility across systems.*

The uv.lock file, rather than loose version constraints on the slide (`pyproject.toml`), provides exact reproducibility info -> must be committed together with the code.

# Ex01: obtain one live frame

```python
"""Core: open the course webcam, preview frames, and close it safely."""

import cv2

CAMERA_ID = 2

def main() -> None:
    camera = cv2.VideoCapture(CAMERA_ID)
    if not camera.isOpened():
        camera.release()
        raise RuntimeError(f"Could not open camera {CAMERA_ID}")

    print(f"Backend: {camera.getBackendName()}")
    print("Press Q in the image window to quit.")
    try:
        while True:
            ok, frame = camera.read()
            if not ok or frame is None:
                raise RuntimeError("Camera read failed")
            cv2.imshow("Camera preview", frame)
            if cv2.waitKey(1) & 0xFF == ord("q"):
                break
    finally:
        camera.release()
        cv2.destroyAllWindows()


if __name__ == "__main__":
    main()
```

might want to change this

```
uv sync  # run only once to setup the environment
uv run labs/lab03/exercises/ex01_camera_preview.py
```

(or click the play / debug button)

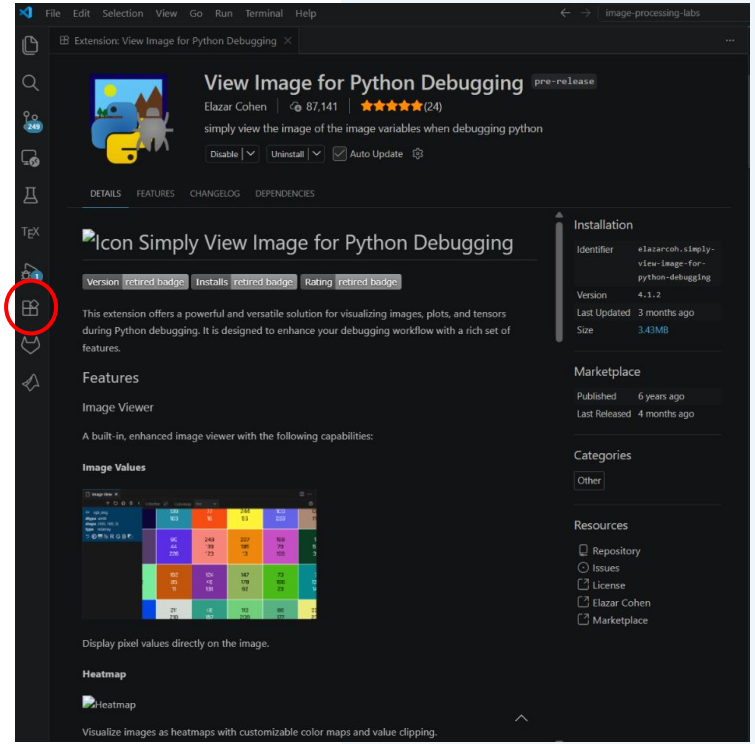
What should happen:

- A live window appears
- The backend name is printed
- Pressing *q* closes the image cleanly

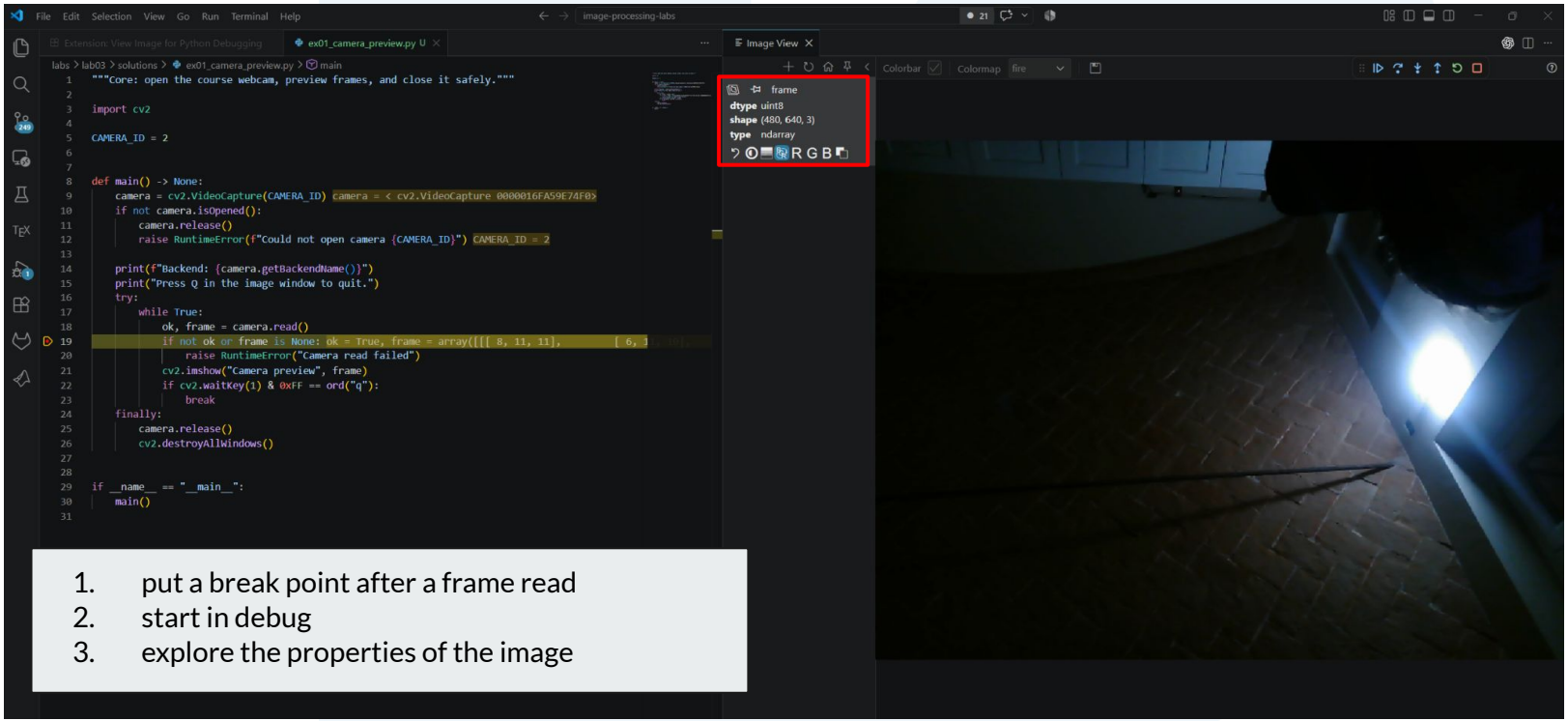
*Take your time to inspect the python code and appreciate how OpenCV is making your life easy.*

For each relevant line write a comment, so that you prove to yourself that you can fully understand everything of this block.

# Wait! How to debug with images?



# Wait! How to debug with images?




1. put a break point after a frame read
2. start in debug
3. explore the properties of the image

# Ex02: Camera properties

In `ex02_camera_properties.py`, you will extend the camera preview so that you can:

- request a capture resolution and fps and inspect the reported result;
- save the displayed frame as a png file when you press S;
- check whether each operation succeeded.

As part of this exercise you can only explore the documentation, see links in the lab README.

and DO NOT use AI or google to search for complete solutions (you already have them).

Navigating the documentation is very quick way to get exposure to lots of opencv concepts.

# Ex02: Camera properties

```python
CAMERA_ID = 1
OUTPUT = Path(__file__).resolve().parents[1] / "outputs" / "camera_frame.png"
REQUESTS = [
    ("width", cv2.CAP_PROP_FRAME_WIDTH, 960.0),
    ("height", cv2.CAP_PROP_FRAME_HEIGHT, 540.0),
    ("fps", cv2.CAP_PROP_FPS, 30.0),
]


def request(camera: cv2.VideoCapture, name: str, property_id: int, value: float) -> float:
    accepted = bool(camera.set(property_id, value))
    actual = float(camera.get(property_id))
    print(f"{name}: requested={value}, set()={accepted}, actual={actual}")
    return actual


def main() -> None:
    camera = cv2.VideoCapture(CAMERA_ID)
    if not camera.isOpened():
        camera.release()
        raise RuntimeError(f"Could not open camera {CAMERA_ID}")
    for name, property_id, value in REQUESTS:
        request(camera, name, property_id, value)

    print("Press S to save one frame, or Q to quit.")
    try:
        while True:
            ok, frame = camera.read()
            if not ok or frame is None:
                raise RuntimeError("Camera read failed")
            cv2.imshow("Camera properties", frame)
            key = cv2.waitKey(1) & 0xFF
            if key == ord("s"):
                OUTPUT.parent.mkdir(parents=True, exist_ok=True)
                if not cv2.imwrite(str(OUTPUT), frame):
                    raise RuntimeError(f"Could not save {OUTPUT}")
                print(f"Saved {OUTPUT}: shape={frame.shape}, dtype={frame.dtype}")
            if key == ord("q"):
                break
    finally:
        camera.release()
        cv2.destroyAllWindows()
```

The three core methods are camera set/get and imwrite.

Remember that a set is a “request to set”, never imperative.
So it’s very important to check if the property was set by querying it again.

Other important settings you can play with:

`cv.CAP_PROP_EXPOSURE` `cv.CAP_PROP_AUTO_EXPOSURE`
`cv.CAP_PROP_GAIN`
`cv.CAP_PROP_FOCUS` `cv.CAP_PROP_AUTOFOCUS`

imwrite returns False on fail for any reasons

# Ex03: not all imwrites are equal!

The file name extension specifies the encoder (i.e. how images are saved on disk)

- BMP
- PNG
- JPG

# Ex03: not all imwrites are equal!

The file name extension specifies the encoder (i.e. how images are saved on disk)

- BMP: uncompressed lossless (very heavy: WxHx3 bytes)
- PNG: compressed lossless (still heavy but less, still retains original quality)
- JPG: compressed lossy (offers tradeoff between quality and size)

# Ex03: not all imwrites are equal!

The file name extension specifies the encoder (i.e. how images are saved on disk)

- BMP: uncompressed lossless (very heavy: WxHx3 bytes)
- PNG: compressed lossless (still heavy but less, still retains original quality)
- JPG: compressed lossy (offers tradeoff between quality and size)

In `ex03_image_io.py`, you will:

- load the png image just acquired from ex02
- save it again in png
- save another copy in jpeg with quality 35 (very high compression)
- reload them and compare them against the first loaded image (use the image difference)
- also compare their size - did the compression work?

# Ex03: not all imwrites are equal!

```python
def image_round_trip(input_path: Path, output_directory: Path):
    frame = cv2.imread(str(input_path), cv2.IMREAD_COLOR)
    if frame is None:
        raise FileNotFoundError(input_path)

    output_directory.mkdir(parents=True, exist_ok=True)
    png_path = output_directory / "still_copy.png"
    jpeg_path = output_directory / "still_quality_35.jpg"
    if not cv2.imwrite(str(png_path), frame):
        raise RuntimeError("Could not write PNG")
    if not cv2.imwrite(str(jpeg_path), frame, [cv2.IMWRITE_JPEG_QUALITY, 35]):
        raise RuntimeError("Could not write JPEG")

    png = cv2.imread(str(png_path), cv2.IMREAD_COLOR)
    jpeg = cv2.imread(str(jpeg_path), cv2.IMREAD_COLOR)
    if png is None or jpeg is None:
        raise RuntimeError("Could not reload an output image")

    difference = cv2.absdiff(frame, jpeg)
    amplified = np.clip(difference.astype(np.int16) * 8, 0, 255).astype(np.uint8)

    print(f"PNG exact: {np.array_equal(frame, png)}, bytes={png_path.stat().st_size}")
    print(f"JPEG mean error: {difference.mean():.3f}, bytes={jpeg_path.stat().st_size}")

    return [
        ("Original BGR", frame),
        ("PNG reload", png),
        ("JPEG reload", jpeg),
        ("JPEG error x8", amplified),
    ]
```

The core methods here are imread and the `IMWRITE_JPEG_QUALITY` flag of imwrite.

# Ex03: not all imwrites are equal!

If you do it with higher jpg quality you may also see no difference!
This is because your video stream can be already compressed by the camera.

Not all cheap cameras can deliver raw images at high frame rate.
Data transfer can be a bottleneck.

# Save a video

```
ok, frame = cap.read()
h, w = frame.shape[:2]

fourcc = cv.VideoWriter_fourcc(*"MJPG")
writer = cv.VideoWriter("data/clip.avi", fourcc, 30.0, (w, h))

if not writer.isOpened():
    raise RuntimeError("Cannot open video writer")

writer.write(frame)
```

- All frames added to the `VideoWriter` must share the same size
- Here the encoder is made explicit: DIVX, XVID, MJPG, X264
- You cannot record audio with OpenCV

*Ex 04: capture a five-second webcam stream and save it to video.*

# Reload the video

```
video = cv.VideoCapture("data/clip.avi")

fps = video.get(cv.CAP_PROP_FPS)
count = video.get(cv.CAP_PROP_FRAME_COUNT)
width = video.get(cv.CAP_PROP_FRAME_WIDTH)
height = video.get(cv.CAP_PROP_FRAME_HEIGHT)

print(f"{width:.0f}×{height:.0f}")
print(f"{fps:.2f} FPS, {count:.0f} frames")
print(f"nominal duration = {count / fps:.2f} s")
```

- You open a video the same way you open a camera
  - you also get frames the same way: `video.read()`
- Not all properties can be `set`, e.g. resolution, but some can, e.g. playback position.

*Ex 05: read a video, write a mirrored copy, and replay it.*

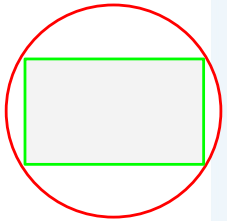
# Estimate camera params

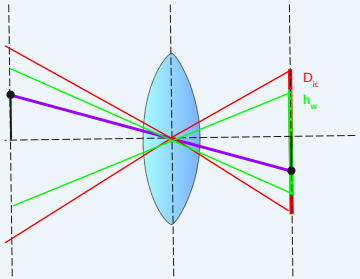
We are going to:

- estimate the FOV of this camera
- estimate its focal length in pixels
  - by knowing the pixel pitch, we can also estimate the focal length in mm

Then we will compare the results with the datasheet.

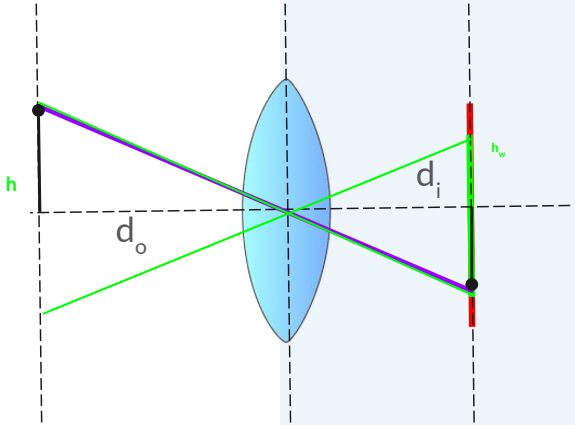
To do this, you will need the measuring tape and the geometry of the thin lens model.



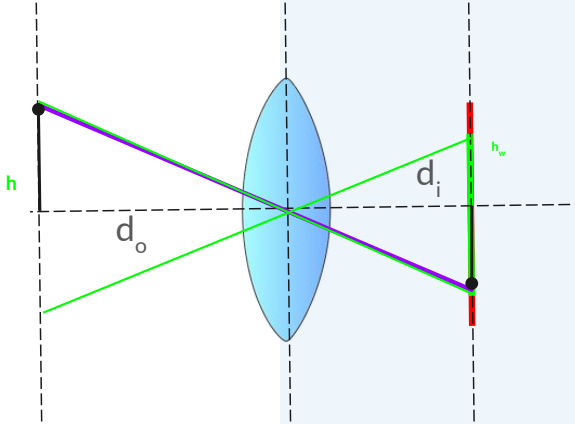



... and some equivalent triangle relations

# Estimate camera params

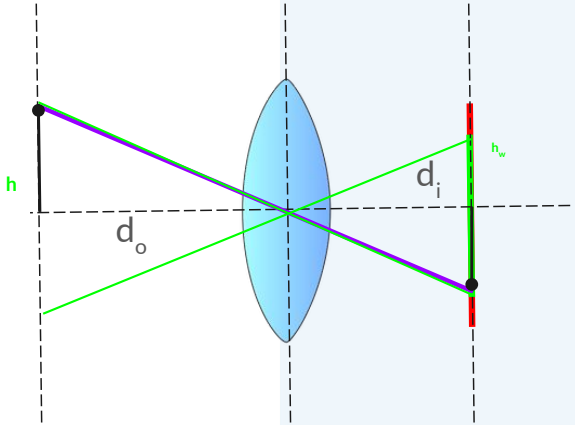


# Estimate camera params




$$\text{FOV} = 2 \arctan\left(\frac{h}{2d_o}\right)$$

# Estimate camera params




$$\mathrm{FOV} = 2\arctan\left(\frac{h}{2d_o}\right)$$

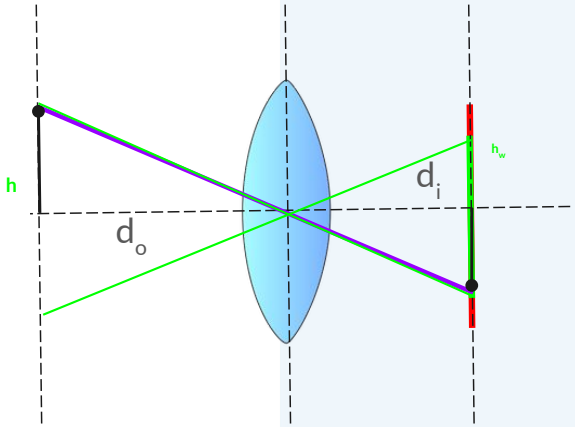
In this relation, the only unknown is $d_i$

$$d_i^{(\mathrm{px})} = \frac{h_w d_o}{h}$$

Next:

- Look for the pixel pitch on the datasheet (google search: “arducam ezBox swift”)
- compute the focal length in mm

# Estimate camera params




You have to recall the thin lens model:

$$\frac{1}{f} = \frac{1}{d_o} + \frac{1}{d_i}$$

notice that since $d_o$ >> $d_i$, you can often approximate f = $d_i$

Compare the focal length you found with the one in the datasheet. Deviation of ~10% to 15% can be explained by:

- uncertainty in the physical location from which $d_o$ should be measured;
- autofocus and focus breathing at short distances;
- radial lens distortion near the image boundaries;
- imperfect target alignment;
- uncertainty in deciding when the target exactly fills the frame;
- approximation of the thin lens model.